

1. Basics of Network Monitoring

What is Network Monitoring?

Network Monitoring is the continuous process of **observing the health, performance, and status of network devices and services** such as routers, switches, firewalls, servers, and even applications.

It's like a **doctor checking the vital signs of a patient** — heartbeat, blood pressure, temperature — but instead, we are checking:

- Is the server up?
- Is the traffic flowing properly?
- Are there any signs of congestion or failure?

Objectives of Network Monitoring

- 1. Availability Monitoring**
Ensure that critical devices are up and reachable.
 - E.g., "Is our main DNS server online?"
- 2. Performance Monitoring**
Measure latency, bandwidth, CPU usage, memory usage, etc.
 - E.g., "Why is the internet slow for the accounting department?"
- 3. Fault Detection**
Detect and respond to hardware or network failures.
 - E.g., "The firewall dropped connection at 3:15 AM – auto-alert sent."
- 4. Security Monitoring**
Detect abnormal traffic or unauthorized access.
 - E.g., "Why is there data being sent to an unknown IP from the server?"

Key Elements Being Monitored

Element	Description
Devices	Routers, switches, firewalls, servers, PCs
Traffic	Data flows, packet counts, bandwidth usage
Services	Web server, email server, DNS, FTP
Resources	CPU, memory, disk usage
Logs	System logs, security logs, error messages

Common Monitoring Activities

- **Ping Checks:** Send ICMP requests to check if a device is online.
- **Port Checks:** Test whether specific services (e.g., port 80 for HTTP) are reachable.
- **Log Monitoring:** Read system logs to find signs of failure or intrusion.
- **Packet Analysis:** Examine raw data packets for troubleshooting or security.

Popular Tools at a Glance

Tool	Use Case
Wireshark	Deep packet inspection
Nagios	Monitoring devices and services
Zabbix	Full-featured monitoring and alerting
PRTG	User-friendly network health visualization
Ping/Traceroute	Connectivity and routing testing

Real-World Scenario: Small Office Network Monitoring

Scenario:

Imagine you're the system administrator for a small company with:

- 30 employees
- 1 firewall
- 2 switches
- 2 servers (file + web server)
- 1 wireless router

Your job is to ensure the network is working efficiently and detect issues before they affect the business.

Monitoring Workflow:

Step	Action
1	Install Zabbix on a central server.
2	Use SNMP to connect to the firewall, switches, and router.
3	Configure checks: ping each device every 30 seconds.
4	Monitor CPU, memory, and disk usage on the servers.
5	Set email alerts for: server CPU > 90%, bandwidth usage > 80%, or device offline.
6	Check Zabbix dashboard daily and review weekly performance trends.
7	If a device goes offline or traffic spikes, investigate using Wireshark to capture packets.

Sample Alerting Scenario

Time: 2:00 PM

Zabbix Alert: "File Server CPU Usage > 90% for 5 Minutes"

✂ Admin Action:

- Logs into the server.
- Sees a backup process is consuming high CPU.
- Reschedules backup job to midnight.

- Adds new threshold: alert if CPU > 80% for 10 min.  Problem solved proactively. No user was affected.

Key Takeaways for Students

- Network Monitoring helps **prevent**, not just **react to**, problems.
- A small amount of automation (alerts, reports) can save a lot of manual effort.
- Knowing **what to monitor** and **how often** is crucial to designing a good monitoring system.

2. Monitoring the Essentials

What Does “Essentials” Mean in Network Monitoring?

In any network, some components are more critical than others. **Monitoring the essentials** means identifying the most important elements of the network and keeping a close eye on their **availability**, **performance**, and **status**.

It’s about **prioritizing** — like a doctor checking the heart and lungs first before moving to minor aches.

Key Essentials to Monitor

Category	What to Monitor	Why It’s Important
Device Health	Uptime, CPU, RAM, Disk	Prevents crashes and overloads
Network Interfaces	Bandwidth usage, errors, status	Detects bottlenecks, failures
Connectivity	Ping responses, traceroute paths	Ensures devices/services are reachable
Critical Services	Web, DNS, email, database	Business functions depend on them
Logs	System, security, application logs	Reveal errors, unauthorized access

Detailed Breakdown

1. Device Availability

- Can the monitoring server **ping** the device?
- Is the device **up or down**?
- Example tools: `ping`, Nagios, Zabbix

2. Performance Metrics

- **CPU** and **RAM** usage — overloaded servers slow down services.
- **Disk usage** — full disks cause crashes or failed backups.
- Tools: `top`, `htop`, Zabbix graphs, PRTG

3. Network Throughput and Errors

- Monitor bandwidth on switches and routers.
- Detect **high traffic**, **packet drops**, or **interface errors**.
- Tools: SNMP queries, SolarWinds, MRTG

4. Service Availability

- Can users access:
 - The company website (port 80/443)?
 - Email server (port 25, 465)?
 - File server (SMB/NFS)?
- Tools: `telnet`, `nc`, Zabbix application monitoring

5. Security Logs and Audit Trails

- Review login attempts, firewall logs, intrusion detection.
- Tools: **Syslog**, **ELK Stack** (Elasticsearch + Logstash + Kibana), **Graylog**

Real-World Scenario: University Network

Scenario:

You're the network admin for a university's computer science department.

Essential systems:

- Web server for student portal
- DNS and DHCP server
- Staff email server
- Three switches and one firewall
- 10 lab PCs used for practicals

What Do You Monitor?

Component	Monitoring Item	Example Tool
Web server	HTTP check, CPU usage	Zabbix HTTP agent
DNS/DHCP	Service availability	Nagios plugin
Email server	Port check (25/465), uptime	PRTG sensor
Switches	Port status, bandwidth usage	SNMP, SolarWinds
PCs	Login attempts, disk space	Remote logging, Zabbix agent

Workflow: Monitoring Essentials in Action

1. **Install monitoring agents** (Zabbix or Nagios) on all servers and critical PCs.
2. **Enable SNMP** on switches/firewall to monitor traffic and interface errors.
3. **Set up periodic ping checks** for all devices every 1 minute.
4. **Create graphs** to visualize bandwidth usage and CPU load.
5. **Set thresholds:** alert if CPU > 85%, disk < 15%, or device is unreachable for 3 checks.
6. **Check dashboards daily**, review alerts, and investigate unusual behavior.
7. **Generate weekly reports** to identify trends or future issues.

Common Mistakes to Avoid

- Monitoring everything equally → Information overload.
- Not setting thresholds → Issues detected too late.
- Ignoring log monitoring → Missed early signs of failure/security breach.

Analogy

Think of your network like a car:

- Engine = core servers (web, mail)
- Wheels = switches and routers
- Fuel = bandwidth
- Dashboard = monitoring dashboard You don't check every bolt daily, but you always check fuel, temperature, and warning lights. That's monitoring the essentials!

Useful Tools Summary

Tool	What It Helps With
Zabbix/Nagios	Device/service monitoring
SNMP	Traffic, interface, device stats
Syslog/Graylog	Log aggregation
Wireshark	Deep packet inspection
Grafana	Visual dashboards
Netdata	Real-time performance metrics

Activity Suggestion (For Classroom)

Lab Task: Set up Zabbix agent on one VM. Monitor CPU, RAM, and web service. Trigger an alert by overloading CPU with a stress tool. Observe how the monitoring reacts.

3. Optimizing the Monitoring Interval

What Is a Monitoring Interval?

A **monitoring interval** is the **time gap between two consecutive monitoring checks** performed by a network monitoring tool.

Example:

- If Zabbix is configured to **ping a server every 60 seconds**, the monitoring interval is **1 minute**.
- If a bandwidth sensor updates every **5 minutes**, that's a **5-minute interval**.

Why Is It Important?

Setting an interval that is too **short** may:

- Overload your network and monitoring server.
- Generate unnecessary logs and alerts.

Setting it too **long** may:

- Cause **delays in detecting** problems.
- Lead to **missed critical events**, such as a service going down and recovering quickly.

Hence, the interval should be optimized based on:

- The **criticality** of the device/service.
- The **network size and load**.
- The **type of check** being performed.

Goals of Interval Optimization

1. **Timely detection** of faults and issues.
2. **Efficient resource usage** (CPU, memory, bandwidth).
3. **Avoiding false alarms** from minor, short-lived issues.

Recommended Monitoring Intervals

Type of Monitoring Target	Suggested Interval	Rationale
Critical services (DNS, mail, firewall)	30 sec – 1 min	Detect failures quickly
Standard servers (file, web)	1 – 5 min	Balance speed and load
Bandwidth/interface stats	5 – 10 min	Changes are gradual
Disk usage, CPU stats	5 min	Stable over short periods
Security logs	Real-time or every 1 min	Fast detection of threats
Non-critical systems	10 – 15 min	Lower priority

Real-World Scenario: Retail Store Chain

Scenario:

You are monitoring 15 branches of a retail store:

- HQ has critical systems: database, ERP, email.
- Each branch has a POS system and local switch.

Interval Strategy:

Component	Importance	Interval
HQ Database Server	Critical	30 sec checks for uptime and service status
POS Terminals	Moderate	5 min checks for availability
Branch Switches	Low	10 min SNMP polling for interface status
Firewall	High	1 min for latency, packet drop checks
Web Server Logs	Security-critical	Real-time syslog monitoring

You also use **log aggregation tools** (e.g., **Graylog**) for real-time events and **Zabbix** for polling-based metrics.

How to Configure Intervals (Examples)

In Zabbix:

- Go to “Items” → set `Update Interval` in seconds.
- Use “Flexible Intervals” (e.g., more frequent during business hours, less during night).

In Nagios:

- Use `check_interval` and `retry_interval` in the configuration.
- Example:

```
check_interval    5    # every 5 minutes
retry_interval    1    # retry every 1 minute if failed
```

In PRTG:

- Choose interval from dropdown for each sensor: 30s, 1m, 5m, etc.

Workflow Example

Task: Monitor a test web server with 3 different intervals: 30s, 1m, and 5m.

Trigger: Stop the web server for 3 minutes.

Compare:

- Which interval detected the problem first?
- Which generated fewer logs?
- Discuss the trade-offs.

Best Practices

1. **Use different intervals for different targets** — not everything needs real-time monitoring.
2. **Use retry logic** — before raising an alert, retry once or twice to confirm.
3. **Implement escalation logic** — repeated alerts should be treated differently than one-time spikes.
4. **Tune based on history** — if issues are rare, increase interval. If frequent, decrease it.

Common Pitfalls to Avoid

- Setting **too frequent** checks for **non-critical systems** → performance overhead.
- Using **same interval for everything** → inefficient and unbalanced.
- Not testing intervals with simulated faults → may miss detecting real issues.

Summary Table

Factor	Consideration
Device Criticality	Higher = more frequent checks
Check Type	Real-time vs statistical trend
Network Load	Avoid excessive monitoring traffic
Alert Sensitivity	Combine intervals with threshold tuning
Business Hours	More frequent checks during active periods

Useful Tools & Features

Tool/Feature	Purpose
Zabbix Flexible Intervals	Time-based dynamic interval
Nagios check_interval	Standard/retry check definitions
PRTG sensor settings	User-friendly interval configs
Prometheus Scrape Interval	Pull-based metrics collection timing
Logstash Pipeline	Real-time log ingestion and transformation

4. Selecting the Right Protocol

What Are Monitoring Protocols?

A **protocol** is a defined method or language used by computers and devices to communicate and exchange information. In network monitoring, protocols define **how** monitoring tools collect and communicate data with devices (like routers, servers, switches).

Choosing the **right protocol** ensures that:

- Monitoring is **efficient and secure**.
- You collect the **correct type of data** (performance, traffic, logs).
- The system is **compatible** with existing infrastructure.

Common Monitoring Protocols

Protocol	Full Name	Used For
ICMP	Internet Control Message Protocol	Checking device availability (e.g., ping)
SNMP	Simple Network Management Protocol	Polling network devices for statistics
HTTP/HTTPS	Hypertext Transfer Protocol	Monitoring web services (web servers, APIs)
SSH	Secure Shell	Accessing logs or running monitoring commands
Syslog	System Logging Protocol	Collecting real-time logs from network devices
WMI	Windows Management Instrumentation	Monitoring Windows systems
NetFlow/sFlow	Network Flow protocols	Analyzing network traffic and bandwidth
NRPE/Agent	Nagios Remote Plugin Executor	Running scripts or checks on remote systems

Protocol Use Cases in Practice

Servers

- Use **SSH** or **monitoring agents** to gather CPU, memory, and disk usage.
- Use **ICMP** to test uptime.

Web Services

- Use **HTTP/HTTPS** to test whether a site or API is reachable.
- Use **URL monitoring** to test specific paths (e.g., /login, /health).

Switches, Routers, Firewalls

- Use **SNMP** to poll:
 - Interface status
 - Traffic rates
 - Packet errors

Bandwidth & Flow

- Use **NetFlow**, **sFlow**, or **IPFIX** to analyze:
 - Who is using the most bandwidth?
 - Where is traffic going?

Logs and Alerts

- Use **Syslog** to stream system logs from Linux, routers, and firewalls to a central collector.

✂ Tool–Protocol Mapping

Tool	Supported Protocols
Zabbix	Agent, SNMP, ICMP, SSH, HTTP, Telnet
Nagios	NRPE, SNMP, ICMP, HTTP
PRTG	SNMP, NetFlow, WMI, Ping, HTTP
SolarWinds	SNMP, NetFlow, Syslog
Graylog	Syslog, GELF
Wireshark	All protocols (packet-level analysis)

Real-World Scenario: Choosing the Right Protocol in a University

Scenario:

You're deploying a monitoring solution for a university IT department.

Infrastructure:

- Cisco routers and switches
- Ubuntu and Windows servers
- Web-based student portal
- Wi-Fi access points
- Firewall appliance
- Email and DNS servers

Protocol Selection Plan:

Component	Protocol	Why
Routers/Switches	SNMP	Gather traffic and interface data
Linux Servers	Zabbix Agent or SSH	Performance and availability monitoring
Windows Servers	WMI or SNMP	CPU, memory, and service status
Web Portal	HTTP/HTTPS checks	Detect downtime or latency
Firewall	Syslog	Receive security logs in real-time
Email & DNS	ICMP, TCP port checks	Ensure service ports are open and responding

Workflow:

1. Enable **SNMPv3** (secure version) on Cisco devices.
2. Install **Zabbix agents** on servers.
3. Configure **Syslog** on the firewall to send logs to a Graylog server.
4. Set up **HTTP probes** for the portal login and health-check URL.
5. Use **ICMP ping** to test availability of all critical systems every 60 seconds.

⚠ Security Considerations

Protocol	Concern	Best Practice
SNMPv1/v2c	Unencrypted	Use SNMPv3 with authentication & encryption
Telnet	Unsecured	Use SSH instead
HTTP	Not encrypted	Prefer HTTPS
ICMP	Can be abused for DoS	Rate-limit ICMP or whitelist sources
Syslog	UDP, unauthenticated	Use TCP + TLS or VPN tunnel if sending over internet

🧠 Protocol Selection Decision Tree

Use a **simple flowchart** to help students decide:

- Is the device a router/switch? → Use SNMP.
- Is it a Linux server? → Use SSH or Agent.
- Is it a Windows server? → Use WMI or SNMP.
- Is it a website? → Use HTTP/HTTPS checks.
- Do you need logs? → Use Syslog or an agent.
- Do you need bandwidth stats? → Use NetFlow/sFlow.

🔧 Suggested Classroom Exercise

Lab Task:

- Pick 3 different systems (e.g., Ubuntu VM, Windows VM, Cisco Router simulator).
- Choose and configure one suitable protocol for each.
- Use a tool (Zabbix/Nagios/PRTG) to monitor a metric.
- Discuss: Why was this protocol the best choice?

✅ Summary Points

- No **one-size-fits-all** protocol — choose based on **device type**, **security**, and **data needed**.
- Use **SNMP** for devices, **agent/SSH/WMI** for servers, **HTTP** for web apps, **Syslog** for logs.
- Always prefer **secure versions** of protocols (e.g., SNMPv3, SSH, HTTPS).
- Test protocol functionality during deployment to ensure compatibility.

5. Setting Thresholds

What Are Thresholds?

In network monitoring, a **threshold** is a **predefined limit or condition** that, when exceeded or violated, **triggers an alert or action**.

Think of it like a car's fuel warning light:

- If fuel < 10%, the light turns on → you are warned in time.
- Similarly, if CPU > 85%, the system may alert you before performance degrades.

Why Are Thresholds Important?

1. **Automation** — alerts and notifications are triggered without manual checking.
2. **Proactive Monitoring** — detect problems early (e.g., rising temperature or disk usage).
3. **Avoiding False Alarms** — only trigger alerts when values exceed *meaningful* limits.
4. **Operational Efficiency** — thresholds help prioritize critical issues over minor ones.

Types of Thresholds

Type	Example	Trigger
Static	CPU usage > 80%	Constant value
Dynamic (baseline)	Network traffic > 150% of average	Based on past data
Time-based	CPU > 90% for 5 minutes	Sustained condition
Multi-condition	CPU > 90% AND Memory > 90%	Multiple parameters
Boolean-based	Service status = “down”	Simple true/false check

Real-World Scenario: Monitoring a University Data Center

Scenario:

You manage the university’s data center, including:

- Web server (Apache)
- Database server (PostgreSQL)
- Network switches
- Application server for online exams

Threshold Examples:

Metric	Threshold	Alert Type
CPU Usage (App Server)	> 85% for 5 minutes	Warning
Disk Usage (DB Server)	> 90%	Critical

Apache Service	Down for > 60 sec	High
Switch Port Errors	> 100 errors/hour	Medium
Failed SSH Logins	> 10 in 10 minutes	Security Alert

Using these thresholds:

- You avoid overloads (CPU/Disk).
- Ensure service availability (Apache).
- Catch security anomalies (SSH brute force).

How Thresholds Work (Workflow)

1. Monitoring system **collects metrics** at defined intervals.
2. Each metric is **compared against its threshold**.
3. If the metric **exceeds or violates** the threshold:
 - An **alert** is triggered.
 - An optional **action** can be executed (e.g., send email, restart service).
4. When the metric returns to normal, a **recovery alert** is sent.

How to Set Thresholds (Examples by Tool)

Zabbix

- Use **Triggers**:

```
{Template OS Linux:system.cpu.util[,user].last()}>85
```

- Add duration check:

```
{...}.avg(5m)}>85
```

Nagios

- Define thresholds in plugins:

```
check_disk -w 80% -c 90%
```

- -w = warning threshold
- -c = critical threshold

PRTG

- Use threshold settings in sensor configuration:
 - Set **warning and error** thresholds.
 - Use **hysteresis** to prevent flapping (e.g., alert only if it stays high for 2 checks).

Lab Activity

Task:

1. Use Zabbix to monitor CPU usage on a VM.
2. Set a threshold: trigger warning at 70%, critical at 90%.
3. Use a stress test tool (`stress-ng`) to raise CPU usage.
4. Observe alert generation and recovery.

Goal: Understand how threshold logic responds to real-time data changes.

Common Mistakes in Threshold Setting

Mistake	Effect
Thresholds too tight (e.g., CPU > 50%)	Frequent false alerts
Too loose (e.g., disk > 99%)	Alert comes too late
No duration buffer	Alerts triggered by brief spikes
Same threshold for all systems	Doesn't reflect varying workloads
Ignoring trends	Doesn't adapt to seasonal/peak behavior

Best Practices

1. **Baseline your environment** — use average/peak data to determine realistic thresholds.
2. **Separate warning vs critical** thresholds.
 - Warning = you have time to act.
 - Critical = immediate action needed.
3. **Use dynamic thresholds** where supported (Zabbix/PRTG).
4. **Review and adjust thresholds** regularly — systems evolve!
5. **Log and escalate** alerts based on severity.

 **Analogy:** Thresholds are like fire alarms in a building.

- Too sensitive: constant false alarms.
- Too insensitive: fire spreads before warning.
- Smart settings: alert only when smoke/heat reaches danger levels.

Summary Table: Threshold Design Strategy

Metric	Baseline	Warning	Critical	Notes
CPU Usage	30–60%	> 70%	> 90%	Consider time duration
Disk Usage	50–80%	> 85%	> 95%	Varies by server
Memory Usage	50–70%	> 75%	> 90%	Combine with swap monitoring
Ping Latency	< 30ms	> 50ms	> 100ms	Depends on distance
Service Availability	Up	NA	Down	Boolean state

6. Network Monitoring Solutions

What Are Network Monitoring Solutions?

Network monitoring solutions are **software systems or platforms** designed to observe the **status, performance, and security** of devices across a network. These tools collect, analyze, and report data about:

- Network traffic
- Device availability
- Service health
- System resource usage
- Log events and security issues

They range from simple open-source tools to complex enterprise-grade platforms.

Key Features of Monitoring Solutions

Feature	Description
Auto-discovery	Automatically detect devices on a network
Dashboards	Visual display of metrics, graphs, maps
Alerting System	Notifies users when thresholds are exceeded
Historical Data	Stores logs and metrics for trend analysis
Integration	Can connect to email, SMS, Slack, ticketing systems
Customization	Flexible monitoring templates and triggers

Categories of Monitoring Solutions

Category	Focus
Infrastructure Monitoring	Monitors servers, services, and device health
Network Flow Monitoring	Tracks bandwidth usage and traffic patterns
Log Monitoring	Collects and analyzes logs (syslog, Windows event logs)
Application Performance Monitoring (APM)	Monitors response time, errors, and user behavior in applications

Popular Network Monitoring Tools (Overview Table)

Tool	Type	License	Suitable For
Nagios Core	Infrastructure	Open-source	Lightweight, customizable setups
Zabbix	Infrastructure + Application	Open-source	Mid to large enterprise use
PRTG Network Monitor	All-in-one	Commercial (free up to 100 sensors)	Easy GUI, good for Windows users

SolarWinds NPM	Enterprise Network Monitoring	Commercial	Large-scale, feature-rich
Prometheus + Grafana	Metrics & Visualization	Open-source	Time-series focused environments
Netdata	Real-time metrics	Open-source	High-frequency local monitoring
Wireshark	Packet Analysis	Open-source	Deep protocol inspection
Graylog	Log Analysis	Open-source/Commercial	Centralized log management
ELK Stack (Elasticsearch, Logstash, Kibana)	Log & Metric Analysis	Open-source	Advanced querying and visualization
LibreNMS	SNMP Monitoring	Open-source	Network-centric monitoring
Icinga 2	Fork of Nagios	Open-source	Modern UI, better scalability

Real-World Use Case Scenario

University Network Monitoring Setup

Environment:

- 150+ devices including:
 - Student lab computers
 - Access points
 - Switches and routers
 - Application servers
 - DNS and email servers

Monitoring Requirements:

- Check server availability and load.
- Monitor Wi-Fi usage and network bandwidth.
- Collect system logs from routers and firewalls.
- Detect security anomalies and unauthorized access.

Tool Selection Plan:

Component	Monitoring Tool	Why
Device health (CPU, memory, disk)	Zabbix	Robust agent support, customizable triggers
Network traffic	ntopng + NetFlow	Specialized traffic insights
Log analysis	Graylog or ELK Stack	Centralized log collection and filtering
Packet inspection (optional)	Wireshark	Deep dive for troubleshooting

Dashboard and alerting	Grafana + Prometheus or Zabbix native	Visualize key metrics
------------------------	---------------------------------------	-----------------------

Workflow:

- 1. Install Zabbix server on a central Ubuntu machine.
- 2. Deploy Zabbix agents on all Linux/Windows servers.
- 3. Enable SNMP on switches and add them to Zabbix.
- 4. Configure alert thresholds (e.g., Disk > 90% = Critical).
- 5. Install Graylog server for Syslog collection.
- 6. Integrate email alerting for critical systems.
- 7. Visualize service health on a 24/7 wall dashboard in the NOC room.

 Suggested Classroom Lab: Multi-Tool Setup

Task:

- Install Zabbix and monitor 3 virtual machines (Ubuntu, Windows, Router simulator).
- Use SNMP and agent-based checks.
- Set a threshold and generate an alert by simulating high CPU.
- Bonus: Set up Grafana to visualize metrics from Zabbix.

Objective: Experience how multiple components and tools collaborate for full-stack monitoring.

 Tool Selection Criteria (How to Choose)

Criteria	Question
Scale	How many devices/systems need monitoring?
Skill Level	Is your team comfortable with CLI or needs GUI?
Budget	Do you prefer free tools or can you invest in commercial options?
Integration	Does the tool integrate with your email, SMS, ticketing, etc.?
Data Type	Do you want log analysis, SNMP, application monitoring, or all?
Real-Time vs Historical	Do you need real-time alerts or trend analysis?

 Summary Chart: Tool Comparison Matrix

Tool	Network	Server	Logs	GUI	Alerting	Scalability
Nagios	✓	✓	Limited	✗	✓	Medium
Zabbix	✓	✓	✓ (basic)	✓	✓	High
PRTG	✓	✓	✓	✓	✓	Medium
SolarWinds	✓	✓	✓	✓	✓	Enterprise
Prometheus	⚠ (via exporters)	✓	✗	✗ (use Grafana)	✓	High
Graylog	✗	✗	✓	✓	⚠ (via plugins)	High

Wireshark	✓ (detailed)	✗	✗	✓	✗	Local
Netdata	✓	✓	✗	✓	✓	Local

Analogy

Think of monitoring tools like medical devices:

- **Zabbix** = a general health monitor (vital signs).
- **Wireshark** = a microscope (packet-level details).
- **Graylog** = a logbook of symptoms.
- **Grafana** = a visual chart for doctors (NOC teams) to analyze trends.

7. Quality of Service (QoS)

What Is QoS?

Quality of Service (QoS) is a set of **techniques and technologies** used to **prioritize certain types of network traffic** over others to ensure **reliable delivery of critical services**, especially under conditions of **limited bandwidth or high network congestion**.

In simpler terms:

- Not all network traffic is equal.
- Some traffic (like video calls, VoIP, or critical database traffic) must be prioritized.
- QoS **ensures that these services perform well**, even if the network is busy.

Why Is QoS Important?

- Ensures Performance of Mission-Critical Applications**
E.g., Video conferencing, VoIP, live streams, etc.
- Minimizes Packet Loss and Latency**
Vital for real-time traffic.
- Improves User Experience**
End-users experience smoother connectivity.
- Supports SLA Enforcement**
Meets performance guarantees in business networks.
- Enables Network Fairness**
Prevents single users/services from hogging bandwidth.

Key QoS Metrics

Metric	Description
Bandwidth	Maximum rate of data transfer (e.g., Mbps)
Latency	Time taken for a packet to travel from source to destination
Jitter	Variation in packet arrival time
Packet Loss	Percentage of packets lost during transmission
Throughput	Actual data delivered per second

QoS Techniques

Technique	Function
Traffic Classification	Identify traffic types (VoIP, web, video)
Prioritization / Queuing	Assign priority levels (e.g., high, medium, low)
Bandwidth Reservation	Allocate minimum guaranteed bandwidth per service
Rate Limiting / Policing	Cap excessive usage
Shaping	Delay traffic to smooth out bursts

Marking	Tag packets for routers/switches to recognize priority (e.g., DSCP, CoS)
---------	--

 Real-World Scenario: University Campus QoS Policy

Scenario:

A university has a 500 Mbps internet link, used by:

- Students (YouTube, social media)
- Online learning systems (LMS, video lectures)
- VoIP (administrative calls)
- Surveillance cameras (IP-based)

QoS Policy Goals:

- Ensure LMS and VoIP never lag.
- Prevent video downloads from consuming all bandwidth.
- Maintain smooth streaming for campus security.

Implementation Strategy:

Traffic Type	Priority	Bandwidth Allocation	Action
VoIP	Highest	10% reserved	Prioritize, low jitter
LMS Servers	High	20% guaranteed	Priority queuing
CCTV Streams	Medium	15% guaranteed	Shaping enabled
YouTube/Netflix	Low	Max 25%	Throttle/download limits
Torrent/FTP	Very Low	≤5%	Rate limited or blocked

 QoS Configuration Workflow

- Classify Traffic:**
 - Identify based on ports (e.g., VoIP on 5060), IPs, or deep packet inspection.
- Mark Packets:**
 - Use **DSCP (Differentiated Services Code Point)** or **802.1p (CoS)**.
- Apply Queuing:**
 - Implement queuing mechanisms like:
 - FIFO (First In First Out)
 - WFQ (Weighted Fair Queuing)
 - LLQ (Low Latency Queuing)
- Apply Policies on Routers:**
 - Use access control lists (ACLs) to match traffic.
 - Apply shaping/policing using router configuration (Cisco, MikroTik, etc.).
- Monitor QoS Metrics:**
 - Use SNMP, NetFlow, or specialized tools to measure effectiveness.

Lab Activity

Objective: Implement a basic QoS policy on a virtual router (e.g., Cisco Packet Tracer or GNS3).

Steps:

1. Create two traffic classes (VoIP and general data).
2. Assign VoIP traffic a higher DSCP value.
3. Apply priority queuing on the simulated router.
4. Generate test traffic using iperf and observe the behavior using Wireshark or built-in traffic monitors.

Tools That Support QoS Monitoring

Tool	Capability
Wireshark	Can view DSCP and CoS packet markings
PRTG	QoS sensor for packet delay, jitter, and loss
Zabbix	Monitors latency, packet loss via ICMP and SNMP
Cisco NBAR	Classifies traffic in real-time for QoS policy
NetFlow/sFlow	View bandwidth usage per application
OpenWRT/MikroTik Routers	Support queuing, shaping, and traffic control

Visualization of QoS Behavior

A simple graph might show:

-  VoIP latency staying below 50ms.
-  LMS bandwidth stable at 20%.
-  YouTube being throttled when others demand more bandwidth.

This is useful to explain QoS effects **visually** in class.

Teaching Analogy

Imagine a university cafeteria line:

- Professors (VoIP) get to skip the line.
- Students (YouTube, FTP) wait their turn.
- Emergencies (LMS exams) are handled instantly.
- The queue is **shaped**, **prioritized**, and **policed** — just like in QoS!

Best Practices for QoS

1. **Always monitor before and after** applying QoS.
2. **Avoid over-prioritization** — not everything can be “critical.”
3. **Classify intelligently** — not just by port, but by behavior (deep inspection).

4. **Test with simulated loads** to fine-tune queues.
5. **Integrate QoS with monitoring** tools to verify its impact over time.

Ping: Checking Basic Network Connectivity

What It Does:

ping checks if a host (e.g., a server or website) is reachable over the network and measures the **time it takes** for data to travel from your machine to the host and back (Round-Trip Time or RTT).

How It Works (Step-by-Step):

- 1. Sends **ICMP Echo Request** packets to a target IP or domain.
- 2. Waits for an **ICMP Echo Reply** from the destination.
- 3. Measures time taken (latency).
- 4. Repeats this for a number of attempts (default is 4 in Windows, infinite in Linux unless limited).

Sample Command (Linux/macOS/Windows):

```
ping google.com
```

Sample Output (Linux):

```
PING google.com (142.250.64.142) 56(84) bytes of data.  
64 bytes from 142.250.64.142: icmp_seq=1 ttl=116 time=21.6 ms  
64 bytes from 142.250.64.142: icmp_seq=2 ttl=116 time=20.8 ms  
64 bytes from 142.250.64.142: icmp_seq=3 ttl=116 time=20.9 ms  
64 bytes from 142.250.64.142: icmp_seq=4 ttl=116 time=21.0 ms  
  
--- google.com ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 20.830/21.104/21.638/0.287 ms
```

Explanation of Each Part:

Part	Description
PING google.com (142.250.64.142)	Target domain and resolved IP
56(84) bytes	56 bytes ICMP payload, 84 total including headers
icmp_seq=1	Sequence number of the packet
ttl=116	Time To Live – number of hops remaining
time=21.6 ms	Round-trip time
4 packets transmitted, 4 received	No loss (good network)
rtt min/avg/max/mdev	RTT statistics: min, average, max, standard deviation

Traceroute: Tracing the Route to a Host

What It Does:

`traceroute` (Linux/macOS) or `tracert` (Windows) shows the path that packets take to reach a destination. It reveals all the **intermediate routers (hops)** and the **delay** at each hop.

How It Works:

- 1. Sends packets with gradually increasing **TTL** (Time-To-Live).
- 2. Each router along the path decreases the TTL by 1.
- 3. When TTL = 0, the router returns an ICMP “Time Exceeded” message.
- 4. This allows mapping the path to the destination.

Sample Command:

```
traceroute google.com
```

(Use `tracert` on Windows.)

Sample Output:

```
traceroute to google.com (142.250.64.142), 30 hops max
 1  192.168.1.1 (192.168.1.1)  1.334 ms  1.243 ms  1.198 ms
 2  10.170.0.1 (10.170.0.1)    10.552 ms  9.821 ms  9.912 ms
 3  isp-gateway.net (203.0.113.1) 15.223 ms 14.921 ms 15.115 ms
 4  142.250.64.142 (142.250.64.142) 21.653 ms 21.718 ms 21.810 ms
```

Explanation:

Field	Description
1, 2, 3...	Hop number (router along the path)
192.168.1.1	IP of the router
1.334 ms 1.243 ms 1.198 ms	RTT for 3 probes (low = good)
Unreachable hop	* * * indicates timeout (packet dropped or blocked)

Other Essential Network Monitoring Commands

Here is a table of additional frequently used network monitoring commands with explanations:

Command	Use Case	Description	Example
<code>ifconfig / ip a</code>	Interface monitoring	View IP addresses, MAC, status	<code>ip a</code>
<code>netstat -tuln / ss -tuln</code>	Port check	Shows listening TCP/UDP ports	<code>ss -tuln</code>
<code>nslookup / dig</code>	DNS troubleshooting	Query DNS records for a domain	<code>nslookup google.com</code>
<code>arp -a</code>	ARP table	View local cache of MAC addresses	<code>arp -a</code>
<code>nmap</code>	Network scanning	Discover hosts and open ports	<code>nmap 192.168.1.0/24</code>
<code>tcpdump</code>	Packet capture	Inspect network traffic in CLI	<code>tcpdump -i eth0</code>
<code>ip route</code>	Routing table	Shows default and static routes	<code>ip route</code>
<code>mtr</code>	Combined ping + traceroute	Real-time route with stats	<code>mtr google.com</code>
<code>ethtool eth0</code>	NIC diagnostics	Check speed, duplex, errors	<code>ethtool eth0</code>
<code>hostname -I</code>	Get local IP	Displays IP address of host	<code>hostname -I</code>
<code>iwconfig</code>	Wi-Fi status	View wireless interface info	<code>iwconfig</code>
<code>vnstat</code>	Bandwidth history	Monitor bandwidth usage over time	<code>vnstat -d</code>
<code>iperf3</code>	Bandwidth test	Measure throughput between two hosts	<code>iperf3 -s, iperf3 -c</code>

Sample Monitoring Workflow in Real Life

Imagine diagnosing slow internet in a university lab:

1. Use `ping 8.8.8.8` to check connectivity.
2. Use `traceroute 8.8.8.8` to identify slow hops.
3. Use `ip a` and `ip route` to verify network configuration.
4. Use `ss -tuln` to check for unexpected open ports.
5. Use `vnstat` or `iftop` to analyze bandwidth usage.
6. Use `tcpdump` or Wireshark for deep traffic inspection.